The Six Loops

A First-Principles Framework for Agentic AI in Human, Startup, Scale-up, and Enterprise Contexts

StepUp.One and Human-Edge.AI

April 2026

Working paper, v1.0

Abstract

This paper proposes a first-principles framework for designing agentic AI systems around the cognitive and operational needs of any viable entity, whether an individual human, a startup, a scale-up, or a large enterprise. The framework rests on four interlocking ideas. First, every viable entity runs six universal loops: sensing, modelling, decision, action, reflection, and memory. Second, each entity has a fitness function, a single question the entity must answer to survive on its own timescale, and this fitness function determines which loops matter most. Third, each loop needs sufficient internal variety to match the environment it is trying to regulate, in the sense of Ross Ashby's Law of Requisite Variety. Fourth, the loops must operate at appropriate tempos and couple with each other tightly enough to function as a single system. Together these four ideas give a complete design brief for any agentic AI system built to serve a specific entity. We apply the framework to four entity types, develop the solo founder selling into enterprise accounts as a detailed case study, and outline implications for how agentic AI should be built and sold in 2026 and beyond. The central claim is that most current AI tooling optimises the one loop that was already working, action, while starving the five loops that actually determine survival. Agentic AI that serves the fitness function rather than the trait is a fundamentally different kind of product, and building it properly requires returning to first principles.

1. Introduction: the problem with adding AI to everything

In the three years since large language models became broadly accessible, the response across the economy has been nearly universal: add AI to everything. Every category of software has spawned an AI version of itself. Every role has been told it must become AI-augmented. Every startup pitching investors has felt pressure to explain how its product uses AI. The volume of AI integration has been extraordinary.

And yet, for all this activity, a strange pattern is becoming visible. The humans using AI most aggressively are not obviously winning. Founders who spend hours a day with ChatGPT still struggle to close deals. Executives who deploy AI copilots across their teams still watch quarterly targets slip. Knowledge workers who write three times faster still report feeling more overwhelmed, not less. Something is not adding up.

The standard explanation is that AI is still early and the tooling will get better. We think the standard explanation is wrong, or at least incomplete. Our claim in this paper is that the problem is not that current AI tooling is too weak. The problem is that current AI tooling is pointed at the wrong thing.

Most AI products sold today, across every category, optimise for faster execution. Write emails faster. Draft code faster. Summarise meetings faster. Produce more output, at higher polish, in less time. This is a real improvement and it genuinely helps. But it helps the one part of any viable entity's cognitive system that was already mostly working. It does almost nothing for the other parts, the parts that actually determine whether the entity survives.

This paper builds a framework for thinking about this properly. The framework is drawn from first principles, not from observation of current AI products. We start by asking what any viable entity must do to survive, regardless of whether AI exists. We identify six universal loops that every entity runs. We show that entities differ in which loops they must prioritise, and that the priority is determined by a concept we borrow from evolutionary biology called the fitness function. We show that each loop needs enough internal complexity to match the environment it is trying to regulate, a concept we borrow from cybernetics called requisite variety. We show that the loops must operate at compatible tempos and couple tightly enough to function as a system, a concept from control theory. And then we apply all of this to four specific entity types, with a detailed case study of the solo founder

selling into enterprise accounts, because this case is concrete enough to stress-test the theory and generic enough for the pattern to transfer.

Our thesis is simple. Agentic AI, the emerging class of AI systems that can run autonomously for minutes or hours and orchestrate multiple tools and skills, is the first class of AI that can serve more than the action loop. It can serve all six. But only if it is designed deliberately, with the fitness function of the entity at the centre and every agent in the system explicitly tied to one of the six loops. Agentic AI built without this discipline will simply accelerate the action loop even harder, and will leave the entity no better off than it was with a fast typing tool.

The stakes are high because of compounding. An entity whose five non-action loops are starved loses small amounts of optionality every day. Those losses compound over months into missed opportunities, into lost deals, into eroded relationships, into survival failures. An entity whose six loops are all matched to its environment compounds in the opposite direction. Over the same months, it accumulates context, sharpens its sensing, deepens its memory, and arrives at decisions its competitors cannot make because they do not see what it sees. Agentic AI makes this second path possible at a cost that was unthinkable even a year ago. The question is whether anyone will build it that way.

The paper proceeds as follows. Section 2 lays out the theoretical foundations, drawing on population genetics, cybernetics, and complexity economics. Section 3 derives the six loops from first principles and explains what each one does. Section 4 develops the fitness function concept and shows how it drives the reordering of loop priorities across entity types. Section 5 develops the requisite variety concept and shows how to measure the gap between an entity's internal complexity and its environment's. Section 6 introduces the tempo and coupling layer. Section 7 applies the full framework to four entity types: the individual, the startup, the scale-up, and the large enterprise. Section 8 presents a detailed case study of the solo founder selling into enterprise accounts, including a diagnostic visualisation and a discussion of how agent augmentation changes the picture. Section 9 discusses implications for building and selling agentic AI. Section 10 concludes.

2. Theoretical foundations

The framework in this paper is not original in its parts. Every idea we use has a serious intellectual lineage that predates the AI era, in some cases by nearly a century. What is new is the combination: using these ideas together to reason about agentic AI design for specific entity types. Before we develop the framework itself, we want to anchor it clearly in its sources, because the provenance matters for anyone who wants to take the framework seriously.

2.1 Population genetics and the fitness function

The concept of a fitness function comes from evolutionary biology, specifically from the mathematical formalisation of Darwin's theory of natural selection in the early twentieth century. Darwin's 1859 work gave us the core insight that organisms vary, variation is heritable, some variants reproduce more than others, and over time this changes populations. But Darwin had no mathematics. He could not say how fast evolution would occur, or how much selection pressure was needed to shift a population, or how to quantify what makes one organism more successful than another.

That mathematical layer was built between roughly 1918 and 1932 by three people who became collectively known as the founders of population genetics. Ronald Fisher's 1930 book, The Genetical Theory of Natural Selection, introduced the idea that you could assign a single number to each genotype representing its expected reproductive success, and that evolution was literally the process by which populations climbed toward higher numbers. Fisher called this number fitness. His Fundamental Theorem of Natural Selection states that the rate of increase in fitness of a population is equal to its genetic variance in fitness. This is the mathematical moment at which the concept of fitness entered science.

Alongside Fisher, J.B.S. Haldane worked out the mathematics of how fast selection could operate, in his 1932 book The Causes of Evolution. And Sewall Wright gave us the visualisation that has shaped every subsequent generation of thinking: the fitness landscape. Imagine a hilly terrain where each point corresponds to a possible configuration of an organism, and the height at that point represents the fitness of that configuration. Evolution is then a population wandering on this landscape, tending to climb uphill. Wright's 1932 paper on the roles of mutation, inbreeding, crossbreeding and selection

in evolution introduced this picture, and it is probably the single most influential metaphor in all of evolutionary theory.

The critical distinction that emerged over the following decades is the distinction between a trait and a fitness function. A trait is a measurable, heritable characteristic of an organism: height, beak length, running speed, skin colour. A fitness function is the question of whether and how that trait contributes to survival and reproduction. The two are not the same. Many traits are byproducts, accidents, or leftovers from ancestors. George C. Williams, in his 1966 book Adaptation and Natural Selection, and later Stephen Jay Gould and Richard Lewontin in their 1979 paper The Spandrels of San Marco and the Panglossian Paradigm, argued sharply against the lazy habit biologists had developed of assuming that every observable trait must be serving fitness directly. A trait, they insisted, is just a feature. Whether it serves fitness is an empirical question that must be asked separately.

This distinction, between a trait and the underlying fitness function it may or may not serve, is the foundation of our entire framework. When we look at an entity and want to understand what it should build into its agentic AI system, we must not start from traits. Starting from traits leads to cargo-cult thinking: Company X succeeded, Company X did this action, therefore we should do this action. Starting from fitness functions leads to structural thinking: what is the real question this entity must answer to survive, and which actions actually serve that question?

The concept escaped biology over the following decades. In the 1960s and 70s, computer scientists built genetic algorithms, which explicitly used fitness functions as scoring rules for candidate solutions to engineering problems. John Holland's 1975 book, Adaptation in Natural and Artificial Systems, is the key text. From there, the concept spread into machine learning, optimisation theory, operations research, and eventually into business strategy, particularly through the work of Stuart Kauffman at the Santa Fe Institute and Eric Beinhocker's 2006 book The Origin of Wealth. When we use the term fitness function in this paper, we are standing on a lineage that runs from Darwin through Fisher and Wright, through Williams and Gould, through Holland and Kauffman, into Beinhocker. It is a real intellectual inheritance, not a metaphor we invented.

2.2 Cybernetics and the Law of Requisite Variety

The second major concept we use comes from cybernetics, the mid-twentieth-century science of control and regulation in complex systems. The key figure is W. Ross Ashby, a British psychiatrist and cybernetician whose 1956 book An Introduction to Cybernetics remains the standard reference. Ashby formulated what he called the Law of Requisite Variety, which states, in its crispest form, that only variety can destroy variety.

In plain language: if the environment you are trying to regulate can take on a hundred different states, and your internal controller can only distinguish ten states, you will lose control ninety percent of the time. You cannot regulate what you cannot perceive and represent. Variety here is a technical term meaning the number of distinguishable states a system can assume, and Ashby's law places a hard mathematical floor on how much internal complexity a controller must have to successfully regulate its environment.

The canonical example is a thermostat. A thermostat has a variety of two: too hot and too cold. This is enough to regulate a room, because the environment-to-be-regulated has low variety (temperature is a one-dimensional continuous variable that the thermostat collapses into two states). But you could not run a hospital with a thermostat, because a hospital has millions of distinguishable states and a two-state controller cannot match them. The hospital's requisite variety is many orders of magnitude higher than the room's, and its control systems must be correspondingly richer.

Ashby's law is not a metaphor or a heuristic. It is a mathematical result from information theory, and it places genuine limits on what any control system can achieve. For our purposes, the law is essential because it tells us exactly when and why an entity will fail to regulate its environment. The entity fails when its internal variety is insufficient for the variety of the environment it is trying to control. Adding more effort does not fix this. Adding more variety does.

This is the core reason agentic AI matters for real-world entities. Agents are, fundamentally, a way of raising an entity's requisite variety without proportionally raising its headcount or its human attention budget. A solo founder trying to regulate the behaviour of fifty enterprise accounts and two hundred stakeholders is facing a variety mismatch. The founder's brain can hold maybe twenty meaningful states at once. The environment has two thousand. The gap is two orders of magnitude. Adding hours to the founder's

week does not close this gap. Adding agents that hold the missing variety does.

2.3 Complex adaptive systems and rugged fitness landscapes

The third tradition we draw on is complexity economics and the study of complex adaptive systems, particularly as developed at the Santa Fe Institute from the 1980s onward. Stuart Kauffman's work is the key bridge here. In his 1993 book The Origins of Order and his 1995 book At Home in the Universe, Kauffman generalised Wright's fitness landscape from biology to any complex adaptive system and introduced the idea of rugged fitness landscapes.

A rugged landscape is one with many local peaks. A population climbing uphill on a rugged landscape can get stuck on a small local peak, unable to reach the much higher global peak that might exist elsewhere. The only way to find the global peak is to occasionally jump to a random new location, accepting a temporary loss of fitness in exchange for a chance at finding a better peak. In biology this is mutation. In business strategy this is pivoting. In individual careers this is changing field.

The rugged landscape idea matters for our framework because it explains why entities must constantly explore, not just exploit. An entity that only exploits its current position climbs to a local peak and stops. An entity that only explores never converges on anything useful. The sweet spot is a balanced cycle of exploitation and exploration, and the balance depends on how rugged the landscape is. In a smooth landscape (few peaks, easy climbing) exploit heavily. In a rugged landscape (many peaks, many traps) explore more.

Eric Beinhocker's 2006 book The Origin of Wealth brings this whole tradition together into a readable form and applies it systematically to economics and business strategy. If any single book is the bridge from Darwin to the boardroom, it is Beinhocker's. Our framework draws on Beinhocker's synthesis throughout.

2.4 Control theory and tempo

The fourth tradition is control theory, the engineering discipline that studies how to design systems that maintain stable behaviour under feedback. The key concept we borrow is the time constant of a system, which is the

characteristic timescale at which a system must update to remain matched to its environment.

A high-frequency trading firm and a pharmaceutical company both have sensing loops. They are not the same sensing loop, because their time constants differ by ten orders of magnitude. The trading firm must sense and respond in microseconds. The pharma company senses and responds over years. A trading firm running its sensing at pharma tempo would be bankrupt in an afternoon. A pharma company running its sensing at trading tempo would exhaust itself chasing noise and go broke within a quarter. The tempo of each loop must be matched to the tempo of the part of the environment it is regulating.

Related to tempo is coupling, which measures how tightly the output of one loop becomes the input of the next. Tightly coupled loops respond to each other quickly. Loosely coupled loops lose information in the handoff. In a small startup, the sensing and decision loops are almost perfectly coupled: the founder hears a customer say something in the morning and pivots by lunch. In a large enterprise, the sensing and decision loops are almost decoupled: a customer insight in January reaches a decision in Q3 of next year, having been compressed into slide after slide along the way. By the time the decision is made, most of the original variety has been lost in the handoffs.

The tempo and coupling dimension is what ties the six loops together as a system rather than as a list of functions. An entity whose six loops run on six different clocks and six different protocols does not have a working cognitive system. It has six isolated activities pretending to be one. The framework must treat tempo and coupling as first-class design considerations, not as afterthoughts.

2.5 Summary of foundations

Our framework draws on four intellectual traditions: population genetics for the fitness function, cybernetics for requisite variety, complexity economics for rugged landscapes, and control theory for tempo and coupling. These traditions span nearly a century and are each backed by serious mathematics and empirical research. When we apply them to agentic AI design, we are not inventing new concepts. We are importing old, well-tested concepts into a domain that has mostly been built on intuition and rapid iteration. This import is the main contribution of the paper.

3. The six loops: deriving the functional anatomy of any viable entity

In this section we derive, from first principles, the minimal set of cognitive and operational functions that any entity must run to remain viable in a non-trivial environment. We claim there are exactly six, and that every agent in any serious agentic AI system must be serving one of them. Agents that do not serve any of the six are either decorative or actively harmful.

The derivation proceeds from a simple observation: every viable entity is, fundamentally, a signal processor. It takes in noisy signals from its environment, builds internal models that compress and predict those signals, decides how to allocate its limited resources, acts in the world, evaluates the outcomes of its actions, and retains the lessons so future cycles can use them. If any one of these functions is absent, the entity cannot respond to its environment. If any one is badly broken, the entity cannot maintain viability over time.

We call these functions loops, not stages, because they are not sequential. They run continuously, in parallel, at different tempos, feeding each other in cycles. Each loop has inputs, outputs, and a fitness criterion of its own. Each can be starved, well-fed, or over-invested. Each can be augmented by agents in specific ways. Let us take them one at a time.

3.1 The sensing loop

The sensing loop is the entity's interface with external reality. Its job is to perceive what is actually happening, as distinct from what the entity wishes were happening or expected would be happening. The sensing loop takes in raw signals, filters out the noise, and surfaces the signals that matter for this entity's fitness function.

The fitness criterion for the sensing loop is time-to-awareness: how quickly does the entity become aware of a change in its environment that matters? An entity with a fast sensing loop notices that a customer is cooling off, or that a regulatory change is coming, or that a competitor has shipped something, within hours or days. An entity with a slow sensing loop notices the same things weeks or months later, by which point the window for response has closed.

Without sensing, every other loop is operating on stale or fabricated data. The modelling loop models a world that no longer exists. The decision loop decides based on fictions. The action loop executes plans that are already obsolete. Memory stores the wrong lessons. Reflection reviews outcomes whose causes were not what the entity thought. Sensing is the ground truth supplier for the entire system, and when it fails, everything downstream fails in ways the entity cannot see.

The sensing loop is the loop most commonly underestimated by busy operators, because sensing feels passive. You are not doing anything when you are sensing. You are just paying attention. Paying attention does not feel like work, so it gets pushed to the margins of the day, done in snatches between meetings, interrupted by notifications. Meanwhile the environment is changing in ways the entity no longer perceives. This is how entities go blind without knowing it.

3.2 The modelling loop

The modelling loop builds and updates internal representations of the world. If the sensing loop supplies raw signal, the modelling loop turns raw signal into structured understanding. It answers questions like: who are my customers, what do they want, how do they decide, what does my competitor's product do, how does my industry actually work, what are the real causes of the outcomes I observe?

The fitness criterion for the modelling loop is predictive accuracy per unit of effort. A good model is one that lets the entity predict the consequences of its actions before taking them, using less effort than it would take to simply try everything. A bad model is one that is either too vague to predict anything, or too specific to predict anything new, or simply wrong in ways the entity has not noticed.

Every entity has models, whether or not the entity is aware of them. A founder who says they do not do strategy is in fact running a strategy, an implicit one, and the implicit strategy is almost certainly worse than any explicit strategy they might bother to write down. The question is not whether to model but whether the models are good, maintained, and tested against reality. Models that are never written down cannot be tested. Models that are never updated go stale.

The modelling loop is where sophistication lives. Two entities facing the same environment with the same sensing capability can diverge enormously based on the quality of their models. One sees a market full of competitors. Another sees the same market as a set of customer segments each with different needs, and recognises that the apparent competitors are actually non-overlapping. The second entity has a richer model and will make better decisions with the same raw signal. This is why intellectual frameworks matter: they are model kits.

3.3 The decision loop

The decision loop allocates resources under uncertainty. It answers questions like: which of the things I could do should I do first? How much should I invest in this bet versus that bet? Whom should I hire? Where should I spend my time today? Which customer should I chase, which should I abandon? The decision loop is where the entity's finite resources get assigned to competing opportunities.

The fitness criterion for the decision loop is expected value per unit of resource committed. A good decision maker chooses actions whose expected upside, properly weighted by probability and by the opportunity cost of the resources they consume, exceeds the alternatives. A bad decision maker chooses actions based on comfort, habit, urgency of the loudest voice in the room, or simply whatever feels productive in the moment.

Decisions are always made under incomplete information. This is not a flaw to be fixed, it is a permanent feature of being an entity embedded in an environment that contains more variety than the entity can perceive. The decision loop must therefore incorporate uncertainty explicitly: what is the confidence interval on this prediction, what is the downside if the prediction is wrong, what is the value of information I could acquire before committing, what is the cost of delay?

The most common failure mode of the decision loop is that it collapses into the action loop. The entity starts doing things without first deciding whether those things are worth doing. Calendars fill up with meetings that were booked because someone asked, not because the entity chose them. Emails get replied to in arrival order rather than priority order. The decision loop is present in principle but has been outsourced to incoming noise. The result is

an entity that is busy but not productive, that fills its time without serving its fitness function.

3.4 The action loop

The action loop produces output in the world. It sends the email, writes the code, makes the call, signs the document, ships the product, gives the speech. The action loop is what an outside observer sees when they watch the entity: the visible evidence that the entity is doing something.

The fitness criterion for the action loop is throughput and quality per unit of cost. A good action loop produces a lot of high-quality output cheaply. A bad action loop is slow, expensive, or produces output that has to be redone. Most of the productivity tooling ever built, from the word processor to the modern AI email assistant, has been aimed at improving the action loop. This is not a coincidence. The action loop is the visible loop, and visible loops attract investment because their improvements are measurable and immediately rewarding.

But here is the central observation of this paper: the action loop is the loop that was already working in almost every viable entity. People know how to send emails. Teams know how to ship code. Companies know how to write proposals. The action loop has been optimised for decades by word processors, CRM systems, collaboration tools, project management software, and now AI copilots. It is not broken. Making it faster produces marginal returns, because the bottleneck is almost never there.

The failure mode of over-investing in the action loop is busy productivity: the entity is producing a lot of output, but the output is not serving the fitness function, because the other loops that would tell the entity what to produce are starved. An AI tool that lets a founder send three times as many emails does nothing if the founder is sending those emails to the wrong people, at the wrong time, about the wrong things, because the sensing, modelling, decision, and memory loops that would tell them the right people, times, and topics are not running.

3.5 The reflection loop

The reflection loop scores outcomes against expectations and updates the rest of the system. When an action produces an outcome, the reflection loop

asks: did this outcome match what we predicted? If not, why not? Was the sensing loop wrong? The model? The decision? The execution? And what should change as a result?

The fitness criterion for the reflection loop is learning rate: how fast does the entity get better at predicting and influencing its environment? An entity with a strong reflection loop extracts lessons from every outcome, updates its models, sharpens its sensing, adjusts its decision criteria, and compounds these improvements over months and years. An entity with a weak reflection loop repeats the same mistakes, because the lessons were never named, and without names the lessons cannot be stored, retrieved, or applied.

Reflection is the loop entities are most likely to skip entirely. It is not urgent. Nobody is waiting for the reflection meeting. The calendar can always be filled with something else. And yet reflection is the loop that determines whether an entity evolves or stagnates. An entity that does not reflect is dead on a long enough timeline, because the environment will eventually change in ways the entity has not noticed, and the entity will not update.

The most common failure of reflection is the comfortable story. When outcomes are bad, the entity reaches for the story that causes the least pain: the timing was wrong, the customer was unreliable, the market was not ready. These stories prevent learning, because they locate the cause of the bad outcome outside the entity, where the entity cannot act on it. Real reflection locates causes the entity can act on, even when the causes are uncomfortable.

3.6 The memory loop

The memory loop retains and retrieves the outputs of all the other loops. Raw signals that the sensing loop detected. Models the modelling loop built. Decisions the decision loop made and the reasoning behind them. Actions taken and their results. Lessons from reflection. All of it must be stored in a form that future cycles can retrieve and use, or each cycle starts from zero and the entity never compounds.

The fitness criterion for the memory loop is retrieval precision per unit of storage. A good memory loop holds a lot of useful context and surfaces the right context at the right moment. A bad memory loop either holds too little (the entity forgets what it learned) or too much without structure (the entity has the information but cannot find it when needed).

Memory is the loop that makes every other loop accumulate over time. An entity with strong memory does not just sense, model, decide, act, and reflect in isolation: it does each of these things while drawing on everything it has previously sensed, modelled, decided, acted on, and reflected about. Each new cycle is richer than the last. This is compounding, and compounding is the single most powerful force available to any entity that wants to improve over long timescales.

Humans are terrible at memory. Our biological memory is biased, lossy, and reconstructive in ways we cannot perceive. We forget ninety percent of what we learn within a month. We remember the emotional residue of events but not the factual detail. We rewrite our memories to be consistent with our current beliefs. This has implications we will return to in Section 7, because it means the memory loop is the highest-leverage loop for individual humans and perhaps the lowest-hanging fruit for agentic AI aimed at individuals.

3.7 Summary: the six loops as an irreducible set

We claim these six loops are the irreducible set: every viable entity must run all six, and no set of five or fewer is sufficient. We arrived at the six by asking what functions are required to take in signals, act on them, and improve over time. Drop any one and the entity cannot function. Drop sensing and you are acting on a fabricated world. Drop modelling and you cannot predict. Drop decision and you drift. Drop action and nothing happens. Drop reflection and you never improve. Drop memory and everything you learn evaporates.

At the same time, we claim no seventh loop is needed. We have looked for candidates. Communication is often proposed, but communication is a sub-function of action (outbound messages) and sensing (inbound messages). Coordination is proposed, but coordination is a specialised case of decision at an organisational level. Strategy is proposed, but strategy is a combination of modelling and decision at a long time horizon. Culture is proposed, but culture is the shared memory plus shared models of a multi-agent entity. None of these require a new loop. All of them decompose into the six.

The six-loop claim is empirical in the sense that it could be falsified. If someone can identify a viable entity that runs only five of the six loops, or a function that is genuinely necessary and does not decompose into the six, the

claim would be wrong. We invite that challenge. Until it comes, we proceed as if the six are complete.

4. The fitness function and why it reorders the loops

In Section 3 we derived six universal loops. In this section we ask a question that cuts the framework much sharper: given that every entity runs all six loops, how does an entity decide which loops deserve the most investment? Why would a solo founder ever prioritise differently from a hundred-year-old enterprise? The answer is the fitness function.

4.1 The fitness function defined, precisely

We define an entity's fitness function as the single question that determines whether that entity survives and propagates, over the timescale that matters to it. The question must be specific, not generic. It must be answerable yes or no, not continuously. And it must be the real question, not a convenient proxy for the real question.

A startup's fitness function is not make more money. Making more money is a consequence, not a cause. The real question is: does this startup find validated demand for a product it can build and sell profitably, before its cash runs out? That is the question on whose answer the startup's existence depends. Everything else is downstream.

A forty-year-old knowledge worker's fitness function is not get more done this week. Getting more done this week is a trait that may or may not serve fitness. The real question is: does this worker compound rare, defensible expertise over decades such that their value per hour rises faster than their industry's average wage growth? That is the question on whose answer the worker's long-term optionality and economic security depends.

A hundred-year-old enterprise's fitness function is not hit quarterly earnings. Hitting quarterly earnings is a trait. Kodak hit its earnings beautifully, right up until it did not exist anymore. The real question is: does this enterprise avoid the slow decay that kills ninety percent of incumbents within twenty-five years of their peak, by noticing the shifts that matter and being willing to cannibalise itself before someone else does? That is the question on whose answer the enterprise's continued existence depends.

4.2 Why the distinction between traits and fitness matters so much

The trait-versus-fitness distinction, which we established in Section 2 as coming from Williams, Gould, and Lewontin's critique of lazy adaptationism in biology, is the single most practical concept in this paper. It is the antidote to cargo-cult strategy and cargo-cult tooling. Almost every serious error in business thinking can be traced back to confusing a trait with a fitness function.

Here is how the confusion works in practice. An observer looks at a successful entity, identifies the traits it displays, and concludes that copying those traits will produce success. But the traits may be spandrels (Gould's term for accidental byproducts) rather than adaptations, and copying spandrels produces no benefit. Or the traits may be adaptations to a different fitness function than the copier faces, and copying them is actively harmful.

When AI tools are sold on the basis of trait optimisation, they are making the same mistake. Write emails faster. Produce content faster. Summarise meetings faster. These are traits. Whether they serve a specific entity's fitness function is an empirical question, and in many cases the answer is no. A founder who is about to miss product-market fit will not be saved by faster emails. A knowledge worker who is about to be replaced by a younger, cheaper worker will not be saved by faster content production. A large enterprise that is about to be disrupted will not be saved by faster meeting summaries. The trait is being optimised. The fitness function is not being served.

This failure is invisible to the people selling the tools and often to the people buying them, because the trait is measurable and the fitness function is not. You can show a founder a chart of how many more emails they sent this week. You cannot easily show them whether they are closer to product-market fit. Measurable wins over meaningful, again and again, and the entity optimises its way into death.

4.3 How the fitness function reorders the loops

Here is where the framework becomes operational. Once you know an entity's fitness function, you can ask which loops serve it most directly. The loops do not all serve the fitness function equally. Some loops are directly connected to the real question. Others are necessary but further removed. The

right priority ordering for an entity's loops is determined by how hard the fitness function pulls on each one.

For a pre-seed startup whose fitness function is find validated demand before cash runs out, the question is essentially an epistemic one: what is actually true about this market and these customers? This means the sensing loop, which supplies ground truth, is the single most important loop. The modelling loop is second, because the startup must translate sensing into a coherent theory of the customer. The decision loop is third, because finite resources must be allocated to the right experiments. Memory is fourth, because every customer interaction is a learning event that must be retained. Action is fifth, downstream of the first four. Reflection is sixth, crucial but lower-leverage at the pre-seed stage because the data is still thin.

For a forty-year-old knowledge worker whose fitness function is compound expertise over decades, the question is essentially about not forgetting. Everything else a knowledge worker does is in service of lasting learning. So memory is the single most important loop. Sensing is second, because learning requires attention to the right inputs. Reflection is third, because reflection is how raw experience becomes structured expertise. Modelling, decision, and action come after.

For a scale-up company whose fitness function is turn a repeatable insight into a repeatable machine, the question is essentially about process entropy: how do you prevent the things that worked at twenty people from breaking at two hundred? The memory loop is first, because institutional memory is what prevents the scale-up from relearning its own lessons every year. The action loop is second, because execution quality at volume matters more than exploration. Decision is third, because decisions now have second-order effects across many teams. Reflection is fourth, because drift is the main threat. Sensing and modelling become specialised functions rather than whole-company activities.

For a large enterprise whose fitness function is avoid slow decay, the question is essentially about undeniable mirrors: how does the entity notice that it is drifting into obsolescence before the drift is fatal? The reflection loop is first, because enterprises die when they stop reflecting on uncomfortable truths. Sensing is second, weaponised against internal blind spots. Modelling is third, explicitly modelling disruption threats. Memory is fourth, but in an

enterprise the problem is too much memory without curation, not too little. Decision and action are already over-tooled in enterprises.

Notice that the same six loops produce four very different priority orderings, and the orderings are not arbitrary. They are derived from the fitness function of each entity. Two entities facing different fitness functions must invest differently in their loops. An agentic AI stack that works for one will waste resources and serve the wrong questions for another. There is no universal agent stack. There are only stacks that serve specific fitness functions for specific entities.

4.4 How to identify a real fitness function

Identifying an entity's real fitness function is harder than it sounds, because entities routinely lie to themselves about what they are doing. They state traits as goals, they mistake proxies for the real thing, and they prefer comfortable fitness functions to uncomfortable ones. Here are three tests we have found useful.

First, the obituary test. If this entity failed completely within ten years, what would be the true cause of death? Not the proximate cause, but the root cause. The answer to this question tells you what the entity was really trying to survive. A startup's obituary would read died of failing to find product-market fit within its runway. An enterprise's obituary would read died of failing to notice the shift that killed it. These causes of death are the dark images of the fitness functions. Whatever causes the death is what the fitness function is trying to avoid.

Second, the trait swap test. Take a trait that the entity claims to care about. Ask: if the entity wildly succeeded at this trait but died anyway, would we be surprised? If the answer is yes, the trait is close to the fitness function. If the answer is no, the trait is a proxy and the real fitness function is elsewhere. Kodak wildly succeeded at photographic film revenue. Kodak died. We are not surprised, because film revenue was never the fitness function. The fitness function was something like maintain dominance of the image capture category across technological transitions, and Kodak failed at that while succeeding at the trait.

Third, the time-horizon test. Different fitness functions operate on different timescales. A high-frequency trading firm's fitness function operates on microseconds. A pharmaceutical company's operates on decades. Name the

timescale on which the entity must succeed or fail. The answer tells you at what frequency the six loops must operate and therefore how to build the agentic systems that serve them.

5. Requisite variety: the mathematics of mismatch

Section 4 established that the fitness function determines which loops matter most. Section 5 establishes how much internal capacity each loop needs, and how to measure the gap between what an entity has and what it needs. This is where Ashby's Law of Requisite Variety does its work, and it is where agentic AI finds its most compelling justification.

5.1 The variety gap, operationally

Every loop an entity runs faces a variety requirement: the number of distinguishable states the environment can assume that the loop must be able to represent and respond to. Every loop also has a variety capacity: the number of states the entity can actually hold in its internal model for this loop. The difference is the variety gap.

When the variety gap is zero, the loop is matched to its environment. The entity can regulate the environment in that loop's domain. When the variety gap is positive, the environment has more variety than the entity can represent, and the entity will lose control in that loop's domain proportionally to the size of the gap. When the variety gap is negative, the entity has more internal variety than the environment requires, which means the entity is wasting effort on distinctions that do not matter.

In practice, variety gaps in real entities are almost always positive, and frequently enormous. A solo founder's sensing loop facing fifty enterprise accounts with ten stakeholders each, each of whom has incentives, career risk, budget pressure, political context, and recent emotional history, is facing an environment with at least two thousand meaningful states. The founder's brain can hold maybe twenty at once. The gap is two orders of magnitude. No amount of effort, no longer hours, no better coffee, closes this gap, because the limit is not effort but cognitive architecture.

An enterprise's reflection loop facing decades of organisational history, thousands of decisions, millions of customer interactions, and hundreds of competitive signals, is facing an environment with billions of meaningful

states. A human-staffed strategy team can hold perhaps a thousand. The gap is six orders of magnitude, which is why strategy teams end up summarising into slide decks that lose almost all the original variety on their way to the executive who makes the decision.

5.2 Why agents are variety multipliers

The most important thing agentic AI does, from this paper's perspective, is raise an entity's requisite variety without proportionally raising its human attention budget. An agent can hold thousands of stakeholder states in an enterprise sales context and surface the right three to the founder in the morning. An agent can track millions of signals across a large organisation's history and flag the patterns that match specific failure modes. An agent can read ten thousand emails and build a warmth map across every account the entity has ever touched.

None of this was possible, at reasonable cost, before 2023. It was possible in principle. You could build a very expensive enterprise system that maintained this kind of variety. But the cost was such that only the largest companies could afford it, and the engineering was such that it would take years to build and months to maintain. Most of that cost and engineering has collapsed. A well-designed agentic system can now hold the variety that would have required a twenty-person team five years ago.

This collapse in the cost of internal variety is the real story of agentic AI. Not faster email. Not cheaper code. Variety. The ability for a solo human to maintain internal models of their environment that match the complexity of the environment for the first time in human history. When you see agentic AI through this lens, the products that matter are the ones that close the variety gap. The products that do not close the variety gap, no matter how clever they seem, are not doing the important work.

5.3 Measuring the variety gap in practice

The framework becomes diagnostic when you measure variety gaps concretely. For a specific entity and a specific loop, the procedure is: first, enumerate the distinguishable states the environment contains that this loop must handle. Second, enumerate the distinguishable states the entity's current internal model for this loop can hold. Third, divide. The ratio is the variety gap.

For a solo founder's sensing loop on fifty accounts: the environment has accounts times stakeholders times meaningful per-stakeholder variables, call it fifty times seven times six, which gives roughly two thousand states. The founder unaided holds twenty. The gap ratio is one hundred to one. This is a brutal number. It means ninety-nine percent of the relevant variety is invisible to the founder at any given moment. Any agent that can close this gap is delivering enormous value even if it does nothing else.

For a forty-year-old knowledge worker's memory loop across a twenty-year career: the environment has maybe twenty thousand lessons, connections, and contextual pieces that could matter for their current work. Unaided human memory reliably holds perhaps five hundred. The gap ratio is forty to one. This is why the most valuable knowledge workers are the ones with unusually strong memory: they are closer to matched on the loop that compounds most.

For a scale-up CEO managing two hundred people: the environment has two hundred people times their current projects times their interpersonal dynamics times their career trajectories, call it ten thousand meaningful states. The CEO's head holds perhaps fifty. The gap ratio is two hundred to one. This is the quantitative version of what CEOs describe qualitatively when they say the transition from twenty people to two hundred feels like losing control. They are literally losing control in Ashby's sense: their internal variety has stopped matching their environment's.

The numbers matter less than the shape of the analysis. In every case the environment has one to three orders of magnitude more variety than the entity's unaided internal model. In every case, agentic AI can, in principle, close most of the gap. And in every case, the entity currently has access to AI tools that do not close the gap at all, because they optimise the action loop rather than raising variety in the loops that actually need it.

6. Tempo and coupling: making the loops run as a system

A system of six loops with correct priorities and sufficient variety is still not a working system if the loops run on incompatible clocks or fail to hand information cleanly to each other. In this section we develop the third

dimension of the framework: the tempo at which each loop must run and the coupling strength between adjacent loops.

6.1 Tempo: the clock speed of each loop

Every loop has a characteristic time constant, determined by how fast the environment it is regulating changes. A sensing loop that responds too slowly misses the signal. A sensing loop that responds too fast drowns in noise. The same applies to modelling (how often must the model be refreshed?), decision (how often must allocations be reconsidered?), action (how often must execution produce output?), reflection (how often must the system score itself?), and memory (how often must the store be updated?).

The tempos of the six loops do not have to match each other. In fact they almost never do. In a solo founder selling to enterprise, the sensing loop must run on an hourly basis because stakeholder sentiment can shift between meetings. The modelling loop runs weekly because account models are stable on that timescale. The decision loop runs daily because time must be reallocated each morning. The memory loop is event-driven, updating every time an interaction happens. Reflection runs weekly, at most. Action runs daily.

When tempos are wrong, the system desynchronises. A sensing loop running too slowly means the model is always operating on stale data and the decisions that follow are late. A reflection loop running too infrequently means lessons from last month's loss are still invisible when next month's similar situation arrives. An action loop running faster than the decision loop that feeds it means the entity is executing plans that have not been thought through. All of these failures look like one-off mistakes from the outside, but they are structural consequences of tempo mismatch.

6.2 Coupling: how tightly loops feed each other

Coupling measures the faithfulness of information handoff between loops. When two loops are tightly coupled, the output of the first becomes the input of the second without loss. When they are loosely coupled, information gets compressed, paraphrased, or delayed in the handoff, and the second loop operates on a degraded version of what the first loop produced.

Small entities tend to have tight coupling by default. A solo founder's sensing and decision loops are coupled by the same human brain: when the founder hears something in a call, the decision happens seconds later because no handoff is required. A co-founding pair is almost as tightly coupled, via real-time conversation. A ten-person team is still tightly coupled if they share space and tools.

Large entities suffer from progressive decoupling as they scale. Information sensed by a customer support representative in Manila takes weeks to reach a strategy meeting in London, loses ninety percent of its specificity along the way, and arrives as a bullet point in a slide deck. By the time a decision is made, the original signal is nearly unrecoverable. This is why big companies get blindsided by shifts that their own staff saw months earlier. The sensing loop was working. The coupling was not.

Agents can improve coupling enormously. Instead of information moving through human chains, where it loses variety at every step, information can move through structured data stores that preserve the original variety. An agent that reads a customer email and updates the knowledge graph makes that email visible to every downstream loop at full resolution. An agent that generates a pre-meeting briefing draws on the original signal, not a summary of a summary of a summary. Coupling that used to be almost impossible at scale becomes the default.

6.3 Why tempo and coupling matter for agentic AI design

Most current AI products treat tempo and coupling as afterthoughts. An agent is bolted onto one loop at one tempo, and whether it synchronises with the other loops in the entity's system is a matter of luck. The result is stacks of AI tools that each work in isolation but do not compound, because they never hand off to each other in the way a working cognitive system requires.

A properly designed agentic system treats tempo and coupling as first-class design decisions. Each agent has a declared tempo: how often it runs, how fresh its data must be, how stale its outputs are allowed to get before they are discarded. Each agent has a declared coupling: which other agents consume its outputs, in what format, at what fidelity. The full system is then a set of loops with known tempos and known couplings, and the designer can reason about whether the system will hold together under real conditions.

This is the difference between a collection of tools and a system. A collection of tools gives you six isolated activities pretending to be one. A system gives you six loops working together as a single cognitive engine that serves the entity's fitness function. The second kind of thing is what agentic AI can uniquely produce, and the first kind of thing is what most current AI products deliver.

7. Four entities, four different stacks

We can now apply the full framework to four entity types and observe how each produces a different loop priority, different variety requirements, different tempos, and therefore different agentic AI stacks. The point of this section is to show that the framework produces concrete, distinct, actionable recommendations rather than a universal template.

7.1 The individual human

An individual human's fitness function, over the timescale of a career, is to compound rare, defensible expertise such that their value per hour rises faster than their industry's average. This is the only fitness function that actually produces long-term economic security for a knowledge worker, and yet almost no individual thinks in these terms.

For this fitness function, the loop ordering is: memory first, sensing second, reflection third, modelling fourth, decision fifth, action sixth. Memory is first because compounding expertise is impossible if the human forgets what they learn, and humans forget ninety percent of what they learn within a month. Sensing is second because learning requires exposure to the right inputs and most humans default to the same inputs for decades. Reflection is third because raw experience does not become expertise without structured review.

An agentic AI system that serves an individual with this fitness function would prioritise: a lifelong memory vault that captures every conversation, article, meeting, lesson, and decision in a searchable, proactive form; a sensing layer that surfaces new inputs worth attending to based on the individual's trajectory; a reflection partner that forces weekly review of decisions and outcomes; and only after all of this, any productivity tool. Current consumer AI gets this ordering exactly backwards.

7.2 The pre-seed to Series A startup

A startup's fitness function at this stage is to find validated demand for a product it can build and sell profitably, before its cash runs out. Every other consideration is subordinate to this question, including team happiness, product elegance, founder health, and even ethical considerations that are not immediately existential. This is not an endorsement of the prioritisation, it is a description of the reality.

For this fitness function, the loop ordering is: sensing first, modelling second, decision third, memory fourth, action fifth, reflection sixth. Sensing is first because a startup is fundamentally an epistemic operation: what is actually true about this market and these customers? Modelling is second because raw sensing must be synthesised into a coherent customer theory. Decision is third because limited resources must be allocated to the right experiments.

An agentic AI system that serves a pre-seed startup with this fitness function would prioritise: continuous sensing across every customer interaction and every adjacent market signal; a live model of target customers that updates with every conversation; a decision-support layer that helps the founder allocate time across competing experiments; a memory substrate that ensures no customer interaction is forgotten; and only then action-loop tools. This is substantially different from what current startup tooling provides.

7.3 The scale-up

A scale-up's fitness function is to turn a repeatable insight into a repeatable machine, which means preventing the unique insights that made the company work at twenty people from dissolving into process entropy at two hundred. The enemy is no longer demand uncertainty. The enemy is internal drift.

For this fitness function, the loop ordering is: memory first, action second, decision third, reflection fourth, sensing fifth, modelling sixth. Memory is first because institutional memory is what prevents the scale-up from relearning its own lessons. Action is second because execution quality at volume matters more than exploration at this stage. Decision is third because decisions now have second-order effects across many teams.

An agentic AI system that serves a scale-up with this fitness function would prioritise: institutional memory systems that preserve the why behind every

decision, not just the what; execution quality tracking across teams and processes; decision frameworks that handle second-order effects; reflection systems that catch drift early; and only then sensing and modelling (which in a scale-up become specialist functions rather than whole-company activities).

7.4 The large enterprise

A large enterprise's fitness function is to avoid the slow decay that kills ninety percent of incumbents within twenty-five years of their peak. The enemy is internal ossification combined with external shift, and the critical danger is that the enterprise's own success prevents it from noticing the shift in time.

For this fitness function, the loop ordering is: reflection first, sensing second, modelling third, memory fourth, decision fifth, action sixth. Reflection is first because enterprises die when they stop reflecting on uncomfortable truths, and the bigger the enterprise the more internal incentives exist to avoid those truths. Sensing is second, weaponised specifically against internal blind spots. Modelling is third, with an emphasis on modelling disruption threats.

An agentic AI system that serves a large enterprise with this fitness function would prioritise: forced reflection mechanisms that hold up undeniable mirrors to the executive team; sensing systems that preserve variety all the way up rather than collapsing it on the way to decisions; modelling systems that explicitly track disruption threats and cannibalisation scenarios; memory curation (the problem in enterprises is too much memory without structure, not too little); and only then decision and action (which are already over-tooled in enterprises).

7.5 The meta-observation

Look at the four loop orderings side by side and notice something: the top loop is the one whose underinvestment kills the entity. Individuals die from forgetting, so their top loop is memory. Startups die from hallucinated demand, so their top loop is sensing. Scale-ups die from process entropy, so their top loop is memory again (but a different kind of memory). Enterprises die from denied reflection, so their top loop is reflection.

This is not a coincidence. It is the fitness function doing its work. The loop that is most starved in the entity's natural state is the loop whose improvement matters most, because that is precisely where the fitness function is pulling hardest. The framework, once you see it, tells you exactly where to invest in any entity you are responsible for. Find the loop whose underinvestment matches the entity's cause of death, and put the agents there first.

8. Case study: the solo founder selling into enterprise

In this section we work through the framework in full detail for one specific entity type: the solo founder trying to sell their pre-seed or seed-stage product into enterprise accounts. We pick this entity because the case is concrete enough to stress-test the framework and because it is also the primary target for our first applied product, Startup Edge, developed as a sibling to this paper.

8.1 The entity and its environment

The entity is one human founder. The environment is approximately fifty target enterprise accounts. Each account contains five to ten stakeholders: the economic buyer, one or two technical evaluators, a champion, a blocker, one or two influencers, and sometimes a procurement gatekeeper. Each stakeholder has their own incentives, career risk, budget pressure, political context, and a recent emotional history with this specific deal. The founder is also managing a target investor list of perhaps thirty names and a working relationship with perhaps ten advisors.

The total environment the founder is trying to regulate contains, on a rough count: fifty accounts times seven stakeholders times five meaningful variables per stakeholder, plus thirty investors times three variables, plus ten advisors. That is just under two thousand distinguishable states that matter for the founder's fitness function. This number will reappear throughout this section because it is the correct Ashby-sense measurement of the variety the founder must regulate.

8.2 The fitness function

The founder's fitness function is: close enough paying customers to validate demand and extend runway before cash runs out. The timescale is three to nine months depending on burn rate and remaining capital. The failure mode has a name: hallucinated demand. A startup dies from hallucinated demand when the founder convinces themselves that something is working when it is not, either because the sensing loop is too weak to detect reality or because the founder's emotional commitment to the idea overrides the signal.

This fitness function pulls hardest on the sensing loop. The founder must see clearly whether each specific deal is actually progressing or whether they are deluding themselves. This is why so many founders lose at this stage: the emotional pressure to believe each deal is close is enormous, and without a ruthless external sensing apparatus, the founder cannot distinguish real signal from wishful thinking. The fitness function also pulls hard on memory, because every stakeholder conversation contains information that will matter for the next conversation, and if any of it is forgotten the deal degrades. It pulls moderately on modelling and decision, and only lightly on action, because the action loop is already well-served by existing tools and is not where deals are won or lost.

8.3 Loop-by-loop diagnosis

The sensing loop. The founder must track which specific humans on each account are moving toward yes or away from it, week by week. The environment requires awareness of two thousand states at hourly tempo. The founder unaided holds twenty. This is the most severe variety gap in the system, at two orders of magnitude, and it runs at a tempo the founder cannot match without agent support. A single agent continuously reading every email, call transcript, and calendar event and maintaining a live warmth map of every stakeholder would close most of this gap.

The modelling loop. The founder must maintain a model of each account: economic buyer, champion, blocker, their incentives, and where this deal sits on their priority list. The environment requires coherent models of fifty accounts. The founder unaided holds maybe five accounts well and the rest as fragmentary impressions. The gap is real but smaller than the sensing gap. An agent that automatically extracts stakeholder roles from email signatures, call

transcripts, and LinkedIn profiles, and builds a structured per-account model that updates with every new interaction, would close most of this gap.

The decision loop. The founder must pick three accounts that get most of this week's time and deliberately let others cool. The environment requires a ranking over fifty accounts based on probability of closure times expected value times time sensitivity. The founder unaided runs on gut feel under pressure. The gap is moderate. An agent that produces a weekly ranked recommendation with explicit reasoning the founder can override would close most of this gap.

The memory loop. The founder must capture every off-hand comment, promise, and objection, and surface the right one before each next meeting. The environment generates dozens of memorable fragments per week across fifty accounts. The founder unaided captures them in scattered CRM notes, emails to self, and Notion pages, losing most detail within forty-eight hours. The gap is severe, and the tempo is event-driven (every interaction). A knowledge graph with proactive pre-meeting brief generation would close most of this gap.

The action loop. The founder must execute emails, calls, demos, and proposals. The environment requires output at daily tempo. The founder unaided is mostly capable, supplemented by ChatGPT for faster drafts. The gap is small or zero. This is the loop that every existing tool is optimising, and it is the loop where further optimisation produces marginal returns. An agent here adds little.

The reflection loop. The founder must name the real cause of each win or loss and update the approach. The environment produces wins and losses at weekly tempo. The founder unaided reflects rarely, and when they do, they reach for comfortable stories that locate the cause outside themselves. The gap is severe but the leverage at pre-seed stage is modest because the data is thin. A reflection agent would help but is not a day-one priority.

8.4 The contrast between today's founder and the agent-augmented founder

One important subtlety must be addressed here. Today's founder is not an AI novice. They have access to ChatGPT, Claude, Cursor, and a dozen other tools. They use AI daily. The relevant contrast is not between a no-AI founder and an AI-augmented founder. It is between a founder using AI as scattered

ad-hoc prompts and a founder whose AI is wired into a structured loop system that serves the fitness function.

Today's founder uses AI in a headless-chicken pattern: one-off prompts for email drafting, occasional use of Claude to summarise a document, sporadic requests to generate outreach copy. Every prompt starts from zero. No persistent state. No feedback into any loop. The result is that the action loop gets ten percent faster and the other five loops are unchanged. The founder feels more productive because action feels productive, but the fitness function is served no better.

The agent-augmented founder uses AI as cognitive infrastructure: continuous stakeholder tracking, a persistent account model, weekly structured decision recommendations, proactive memory surfacing, and disciplined post-deal reflection, with the action loop running downstream of all of them. The founder is not faster at anything individually. The founder is regulating a much larger environment with the same forty hours. The variety gap closes. The fitness function is served.

This contrast is the single most important insight for anyone building or buying AI tools for startups. It is not about whether to use AI. It is about whether the AI is plugged into a structured system or scattered across random prompts. The former is transformative. The latter is marginal. Most current tools, even excellent ones, are the latter.

8.5 What this implies for product design

Everything above implies a specific shape for a product serving this entity. The product must have memory and sensing as its substrate, with the other loops built on top. It must expose a single surface to the founder: a daily briefing that says do these fifteen things today, with reasoning, ready to approve and execute. It must refuse to become a dashboard, because dashboards are what founders flee when they are drowning, not what they flee to. It must measure its success by briefing completion rate, not by number of features used. It must learn from every approved and rejected action, because the founder's rejections are the clearest signal about what is actually noise in their specific environment.

This is a radically different product from anything currently sold to founders. Current tools are dashboards, CRMs, sales engagement platforms, and note-taking apps. They are all action-loop optimisers sold in the language

of productivity. The product that serves the founder's actual fitness function is a decisions engine sold in the language of survival. That is what Startup Edge is being built to be, and the sibling product PRD develops the operational details.

9. Implications for building and selling agentic AI

The framework we have developed has practical implications for anyone building or selling agentic AI in 2026. We close out the substantive content of the paper by naming the five we think matter most.

9.1 Sell outcomes, not outputs

The action loop is the outputs loop. Every tool sold on the promise of more output, better output, or faster output is selling action-loop optimisation. These tools will continue to sell, because outputs are measurable and marketable, but they will not produce outsize results for their buyers, because outputs are the loop that was already working. The tools that produce outsize results will be the ones that sell outcomes, and outcomes require serving the loops that outputs never touched.

The pitch for an outcomes tool is fundamentally different from the pitch for an outputs tool. An outputs tool says: do what you were already doing, faster. An outcomes tool says: do fewer, better things, because I have already figured out which ones matter. The second pitch is harder to sell because it requires the buyer to trust that the tool has made the right call. But it is the pitch that actually serves the fitness function, and the buyers who believe it will be the most valuable customers any agentic AI company acquires.

9.2 Pick a fitness function, not a persona

Product teams conventionally segment by persona: founder, investor, executive, individual contributor. Personas are trait groupings, and trait groupings mislead because they gather people with very different fitness functions under one label. A pre-seed founder and a Series C founder are both founders, but their fitness functions are different (the first is searching for demand, the second is scaling operations) and their loop priorities are correspondingly different. A product designed for the persona serves neither fitness function well.

The framework suggests segmenting by fitness function instead. Build for pre-seed-founders-searching-for-demand, or for Series-C-founders-scaling-operations, or for knowledge-workers-compounding-expertise, or for enterprises-avoiding-decay. These are narrower segments and therefore smaller markets, but within each segment the product can be designed to serve the exact fitness function, which is what produces outsize results and genuine retention. A broader product serving no specific fitness function retains no one.

9.3 The Managed Agents moment

Anthropic's launch of Claude Managed Agents in public beta on April 8, 2026, six days before this paper was written, changes what is economically feasible for a small team to build. Before Managed Agents, a product that required long-running reliable agentic sessions faced three to six months of sandboxing, checkpointing, state management, and orchestration infrastructure before a single line of agent logic could be written. Now it is an API call.

This collapse in infrastructure cost is what makes the framework in this paper newly actionable. A solo founder can now build an agentic system that serves another solo founder's six loops, using infrastructure that was available only to the largest enterprises eighteen months ago. The ability to write a thirty-page agentic specification and have it execute reliably as a three-hour session was literally impossible a week before this paper. Now it is a standard API call against a documented service.

The implication is a window. For perhaps twelve to eighteen months, the teams that internalise the framework early will have a structural advantage over teams that are still building action-loop optimisers. After that window, the framework will become common knowledge, the tools will become table stakes, and the competitive advantage will shift to execution quality and distribution. But in the window, the framework is a genuine moat.

9.4 Measure loops, not features

Conventional product metrics measure features: how many users, how many sessions, how much time in app, how many actions taken. These are output metrics, which is to say they measure the action loop. A product built on the six-loop framework should measure loops: is the sensing loop closing

the variety gap this user faces? Is the memory loop surfacing the right context at the right moment? Is the decision loop producing rankings that track with real outcomes?

Loop metrics are harder to collect than feature metrics, because they require measuring the entity's outcomes, not the product's outputs. But they are the right metrics, because they are the ones tied to the fitness function. A product that ranks highly on feature metrics and poorly on loop metrics is a product that is making its users feel busy without serving their survival. A product that ranks highly on loop metrics and modestly on feature metrics is a product that is serving its users' fitness functions and will retain them for the long term.

9.5 Build the platform, but sell the expert

A final implication concerns go-to-market. The framework is abstract. Founders, executives, and individuals do not buy frameworks. They buy specific help with specific problems, delivered by specific humans. The most successful application of this framework in practice will be the combination we call Playbook plus Platform plus Expert: a battle-tested playbook for a specific fitness function, a custom agentic platform built to execute that playbook, and a real expert who runs the platform on behalf of the user and interprets the results.

This is how StepUp.One and Startup Edge are related: StepUp.One is the done-for-you expert service, Startup Edge is the do-it-yourself platform, and the playbook is the same in both cases. The expert runs the platform, generates outcomes, and over time teaches the user to run the platform themselves. The user graduates from expert-delivered to self-serve when they are ready. Most users never graduate, because the expert is genuinely valuable beyond what any platform alone can provide. The ones who do graduate become reference customers for the platform.

This model is not new. It is how enterprise software has always been sold: the product plus the services team plus the playbook. What is new is applying it at the individual and small-startup level, where the economics only work because agentic AI has collapsed the marginal cost of delivery. For the first time in history, you can sell playbook-plus-platform-plus-expert to a pre-seed founder at a price the founder can afford and a quality the founder could never produce unaided. This is what Human-plus-AI-equals-outcomes looks like in

practice: not AI replacing humans, not humans using AI as a faster typewriter, but humans and AI combined into a single cognitive system that serves the human's fitness function.

10. Conclusion

We began this paper with an observation: the humans using AI most aggressively are not obviously winning. Our explanation is that current AI tooling optimises the wrong loop. It makes the action loop faster while starving the five loops that actually determine whether an entity survives and compounds. Fixing this requires returning to first principles and asking what any viable entity needs to do to survive, independent of whether AI exists.

The answer is six loops: sensing, modelling, decision, action, reflection, and memory. Each has a specific function, a specific fitness criterion, and a specific failure mode. Every viable entity runs all six, and no subset is sufficient. Current AI tools mostly serve the action loop alone, which is why their benefits are real but marginal.

The loops do not all matter equally for every entity. The fitness function, a concept borrowed from population genetics, determines which loops matter most for a specific entity over its specific timescale. Individuals compound or forget, so memory leads. Startups find demand or die, so sensing leads. Scale-ups preserve process or drift, so memory leads again in a different form. Enterprises reflect or decay, so reflection leads. The framework produces four different stacks for four different fitness functions, and the differences are structural, not cosmetic.

Each loop needs enough internal variety to match the environment it regulates. Ross Ashby's Law of Requisite Variety, from cybernetics, sets a hard mathematical floor on how much internal complexity a controller must have. In real entities the gap between required variety and actual variety is typically one to three orders of magnitude, and agentic AI is the first technology capable of closing such gaps at a cost individuals and small teams can afford. This is the real story of what agentic AI enables. Not faster typing. Variety matching.

The six loops must also run at appropriate tempos and couple tightly enough to function as a system. Tempo and coupling failures produce entities with the right components arranged incorrectly, working in isolation, unable

to hand information to each other. Properly designed agentic systems treat tempo and coupling as first-class design considerations rather than afterthoughts.

When we apply the full framework to a specific entity, the solo founder selling into enterprise accounts, the diagnostic is sharp. The founder has severe variety gaps in sensing and memory, moderate gaps in modelling and decision, and essentially no gap in action. The fitness function pulls hardest on sensing. Today's founder is already using AI but the use is scattered and confined to the action loop, which is why so many founders feel productive without closing deals. An agent-augmented founder who runs a structured system across all six loops has a fundamentally different relationship with their environment and their fitness function.

The implications for building and selling agentic AI follow naturally. Sell outcomes, not outputs. Segment by fitness function, not by persona. Exploit the twelve-to-eighteen month window opened by Claude Managed Agents and similar infrastructure. Measure loops, not features. Combine platform with expert delivery to make the economics work at the individual level.

We close with the thesis that has run through the entire paper and will run through everything we build on top of it: Human plus AI equals Outcomes. AI alone produces outputs. Humans using AI as a typewriter produce outputs more efficiently. Humans combined with AI in a structured cognitive system that serves a specific fitness function produce outcomes no humans and no AI could produce separately. This is the equation that matters. Everything we are building, Startup Edge, Human-Edge.AI, StepUp.One, is an attempt to make this equation true for one more entity at a time.

The framework in this paper is a starting point, not a finished theory. We expect it to be revised as we apply it, refined as we build products on top of it, and extended as we encounter entities it does not yet serve well. We publish this version to make the theory legible to the people we are working with, so they can criticise it, contribute to it, and hold us accountable to its implications. The framework earns its place only if it produces better products, clearer thinking, and surviving entities. We will know in a year whether it does.